

PyTorch Buildings GUI

Documentation

*Architecture, simulation behavior, persistence, method contracts, diagrams,
development rules, and simulation value references.*

Contents

Contents	2
Scope	3
Application Purpose	3
Existing Flowchart	4
Architecture	5
Layers	5
State Ownership	5
Main Window Composition	6
Core Design Decisions	6
Canvas Mutations and Undo	6
Connections Are Validated Data Mappings	7
Editing Is Locked During Simulation	8
Zone Count Is Project-Wide	8
Stable IDs Protect Saved Connections	8
Simulation Data Flow	9
Time and Tensor Contracts	9
Node Ordering	9
Input Precedence	9
Stop and Resume	10
External Input Data	11
Saved Project Format	11
Method Return Conventions	12
Method Reference	12
src/main.py	12
src/models/building_model.py	13
BuildingModel	13
src/simulation/input_data.py	13
src/simulation/runner.py	14
SimulationRunner	14
SimulationThread	15
src/simulation/plotter.py	15
src/gui/canvas_items.py	15
ComponentItem	15
src/gui/interactive_canvas.py	16
Component operations	16
Connection operations	16
src/gui/canvas_tool_manager.py	17
src/gui/file_manager.py	18
src/gui/dialogue_manager.py	19
src/gui/main_window_project.py	19
src/gui/main_window_simulation.py	20

src/gui/main_window_ui.py	20
src/gui/state_manager.py	21
src/gui/undo_commands.py	21
src/gui/plot_widget.py	22
src/gui/plot_containers.py	22
Simple GUI Modules	22
Current Constraints	23
Appendix: Simulation Value Reference	24
Inputs, Setpoints, and Temperatures	24
Airflow, Pressure, and Controller State	25
Power and Heat Flow	26

Scope

The bundled `neuromancer_repo/` numerical library is outside this document's method-reference scope. This document explains how `src/` configures and calls that backend.

Many source docstrings currently use placeholders such as `Summary: Run` or `Returns: Return the computed value`. The reference below states the actual contract visible from the implementation. When code and documentation disagree, the code is authoritative and this document should be updated.

Application Purpose

The application is a PyQt6 desktop editor for building and simulating an HVAC system. A user can:

1. place supported HVAC components on a canvas;
2. edit component and project parameters;
3. connect compatible component signals;
4. save and load project layouts;
5. run, stop, resume, and reset a NeuroMANCER simulation; and
6. inspect and export simulation plots.

The supported component types are:

Component	Responsibility
<code>SolarGains</code>	Generates outdoor temperature and solar-gain inputs
<code>RTU</code>	Models rooftop-unit supply temperature, pressure, airflow, and power
<code>VAVBox</code>	Models zone airflow, reheat, and supplied heat flow
<code>Envelope</code>	Models zone temperatures and building thermal behavior

Existing Flowchart

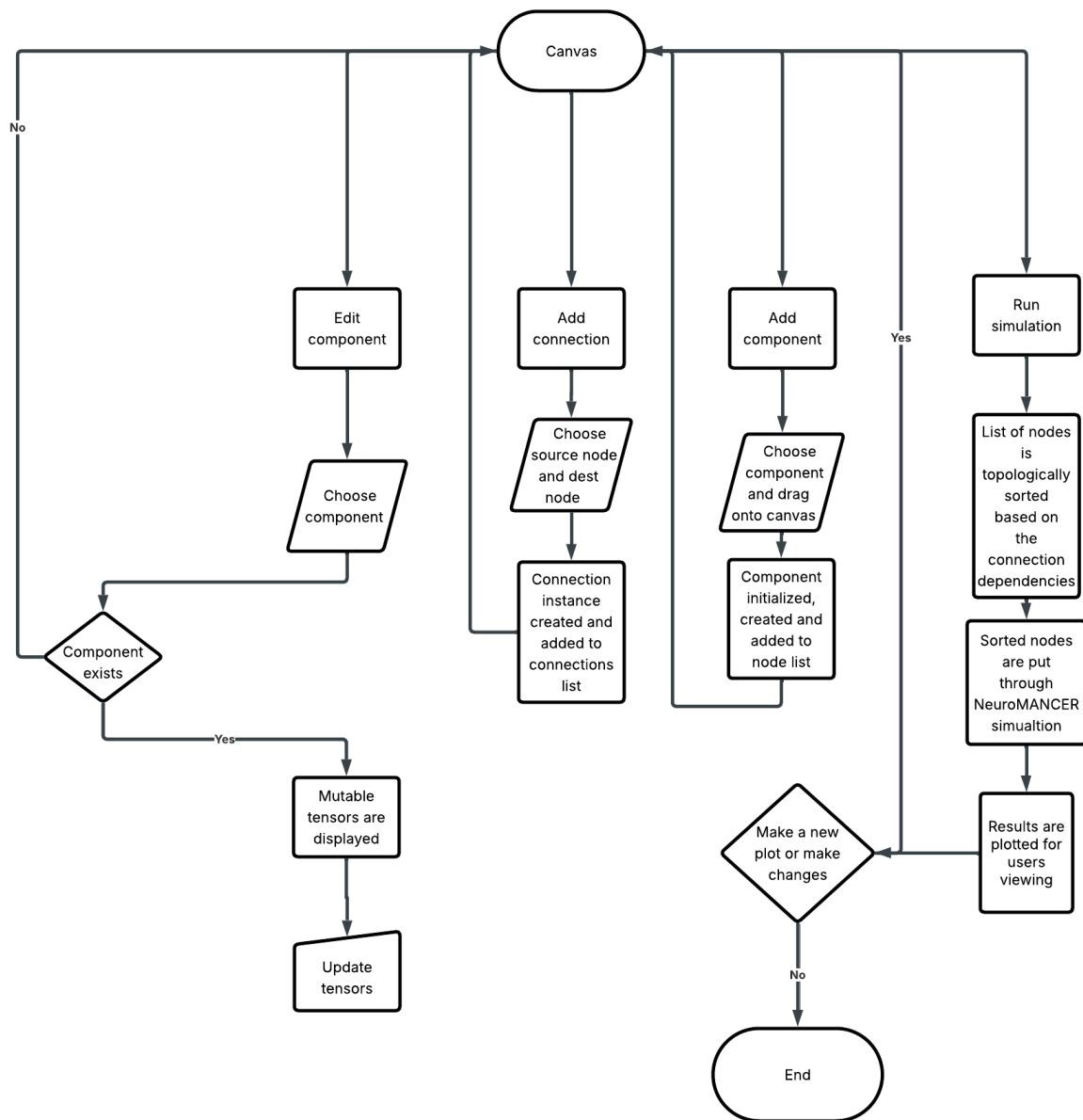


Figure: Basic Simulation Flow

Architecture

Layers

Layer	Main location	Responsibility
Entry point	src/main.py	Starts Qt and displays the main window
GUI and editing	src/gui/	Canvas, dialogs, project workflows, undo/redo, plots, and UI state
Project model	src/models/building_model.py	Project-level state and simulation settings
Simulation adapter	src/simulation/	Converts project state into backend inputs and executes simulations

`BuildingModel` is the authoritative project-level model. `InteractiveCanvas` is the visual editor and the mutation boundary for component and connection operations. `SimulationRunner` translates the live model into the keys, tensors, node order, and calls required by NeuroMANCEr.

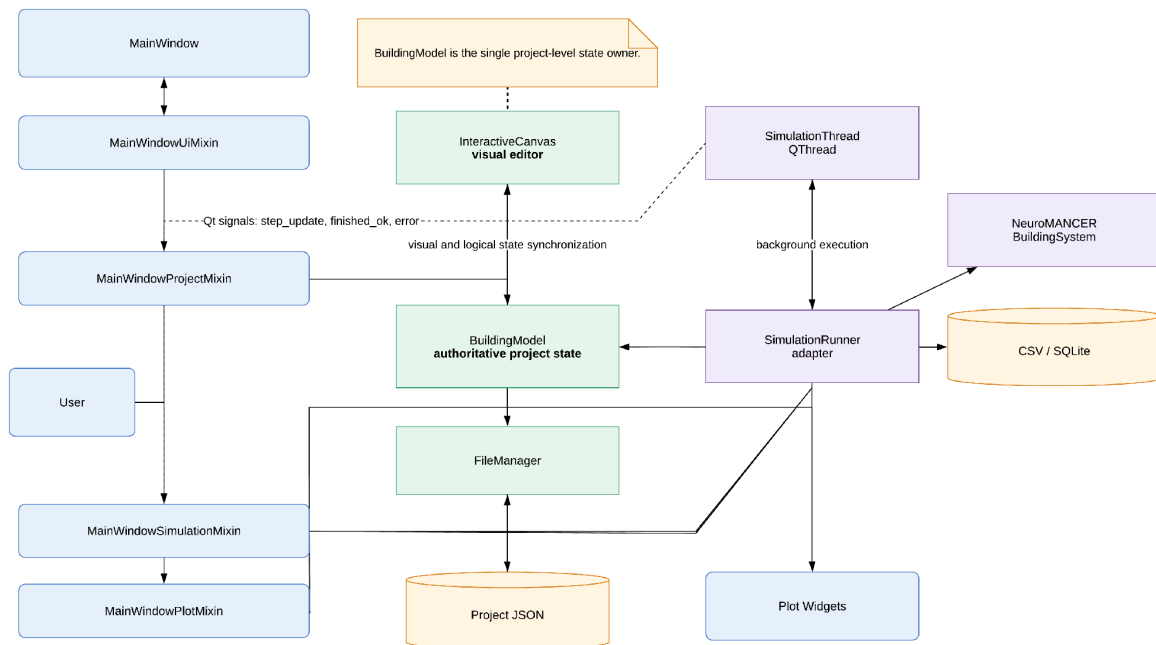


Figure: System Architecture

State Ownership

`BuildingModel` owns:

- project name and zone count;
- simulation timing;

- component items and their nodes;
- logical connections;
- optional external input-data path and summary; and
- optional control-policy overrides.

`InteractiveCanvas` owns:

- the `QGraphicsScene`;
- component graphics;
- visual connection lines and arrows;
- editing modes, previews, selection, pan, and zoom; and
- undoable canvas mutations.

`ComponentItem` intentionally combines three concerns that must stay synchronized:

7. a visual canvas item;
8. a configured NeuroMANCER HVAC component; and
9. a `BuildingNode` used by the simulation backend.

This coupling makes interactive editing immediate. It also means that mutations must update both the visual scene and `BuildingModel`, which is why user-visible mutations go through `InteractiveCanvas`.

Main Window Composition

`MainWindow` is composed from focused mixins:

Mixin	Responsibility
<code>MainWindowUiMixin</code>	Constructs panels and reports setup issues
<code>MainWindowProjectMixin</code>	Project editing, dirty state, and save/load
<code>MainWindowSimulationMixin</code>	Simulation lifecycle and progress
<code>MainWindowPlotMixin</code>	Plot configuration and export

The mixins operate on one `MainWindow` instance. They divide workflow code without creating additional state owners.

Core Design Decisions

Canvas Mutations and Undo

Public canvas mutation methods push a `QUndoCommand`. Their corresponding `internal_*` methods perform the actual change. Undo and redo restore the original object instances instead of constructing replacements.

This matters because connections, graphics items, stable IDs, and callbacks may reference the original object. Bulk deletion is represented by one command so the entire removed subgraph is restored as one user action.

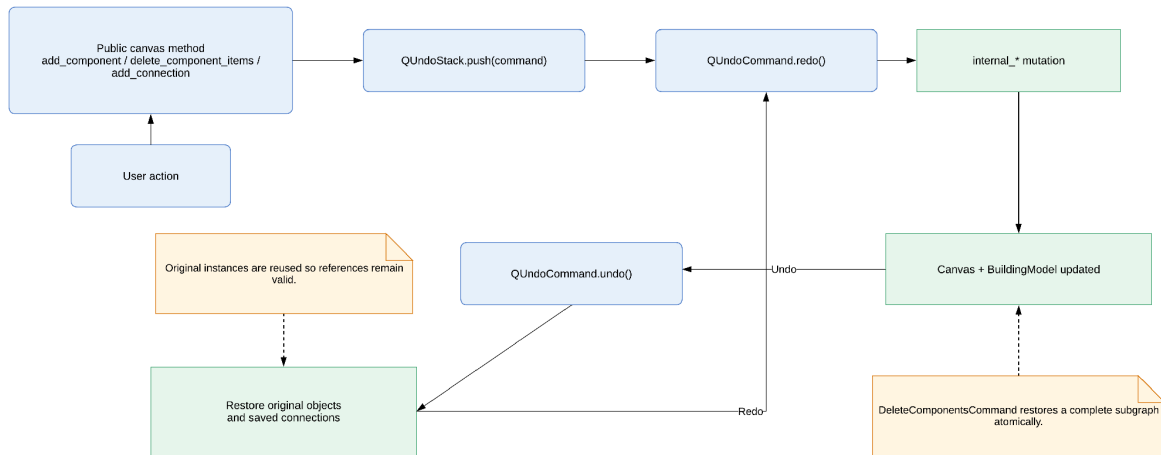


Figure: Undo and Redo Command Flow

Connections Are Validated Data Mappings

A connection is not only an arrow. It stores one or more mappings such as:

```
rtu.T_supply -> T_supply_upstream
rtu.P_supply -> P_duct
```

`HVAC_CONNECTIONS` is an allowlist. Unsupported component pairs are rejected because arbitrary edges could map outputs into inputs that the backend cannot consume. Before each simulation, the runner rebuilds node input maps from current connections so removed connections cannot leave stale backend mappings.

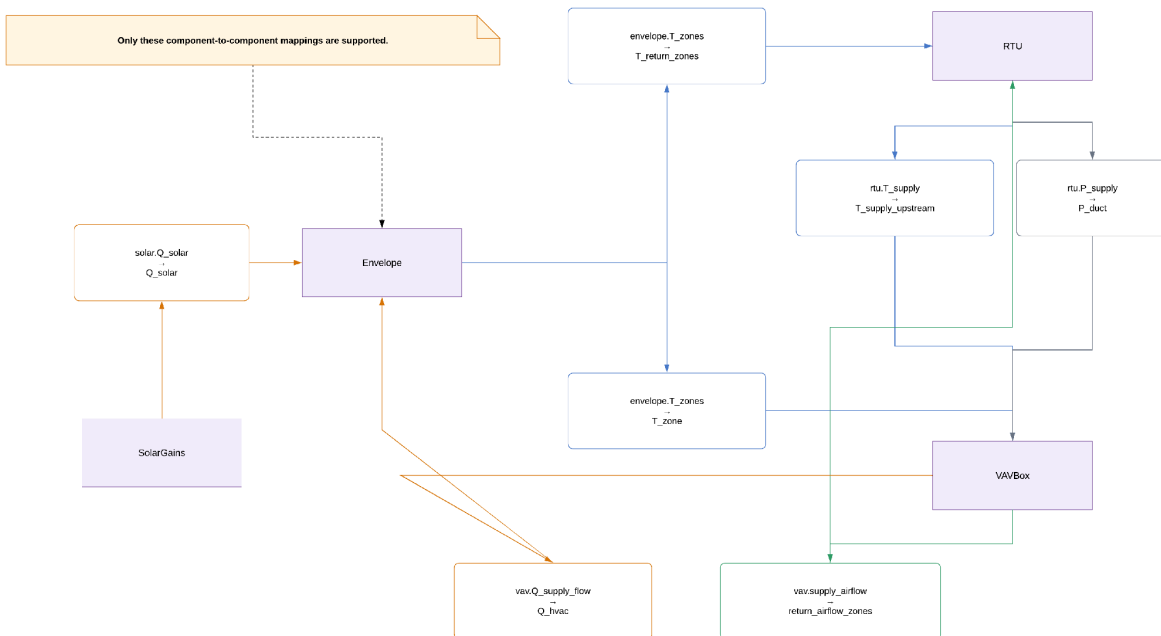


Figure: HVAC Signal Map

Editing Is Locked During Simulation

Simulation uses the live model and component objects. Structural edits during a rollout could change nodes, parameters, or input maps partway through execution. The application therefore disables editing during simulation and invalidates cached results whenever the project changes.

Zone Count Is Project-Wide

The zone count is owned by `BuildingModel`. Zone-dependent component vectors and matrices are resized when it changes. Existing entries are preserved and new entries are zero-filled.

Zero-filling is conservative: the application should not invent physically meaningful resistance, capacitance, airflow, or adjacency values. Setup warnings identify values that require user configuration.

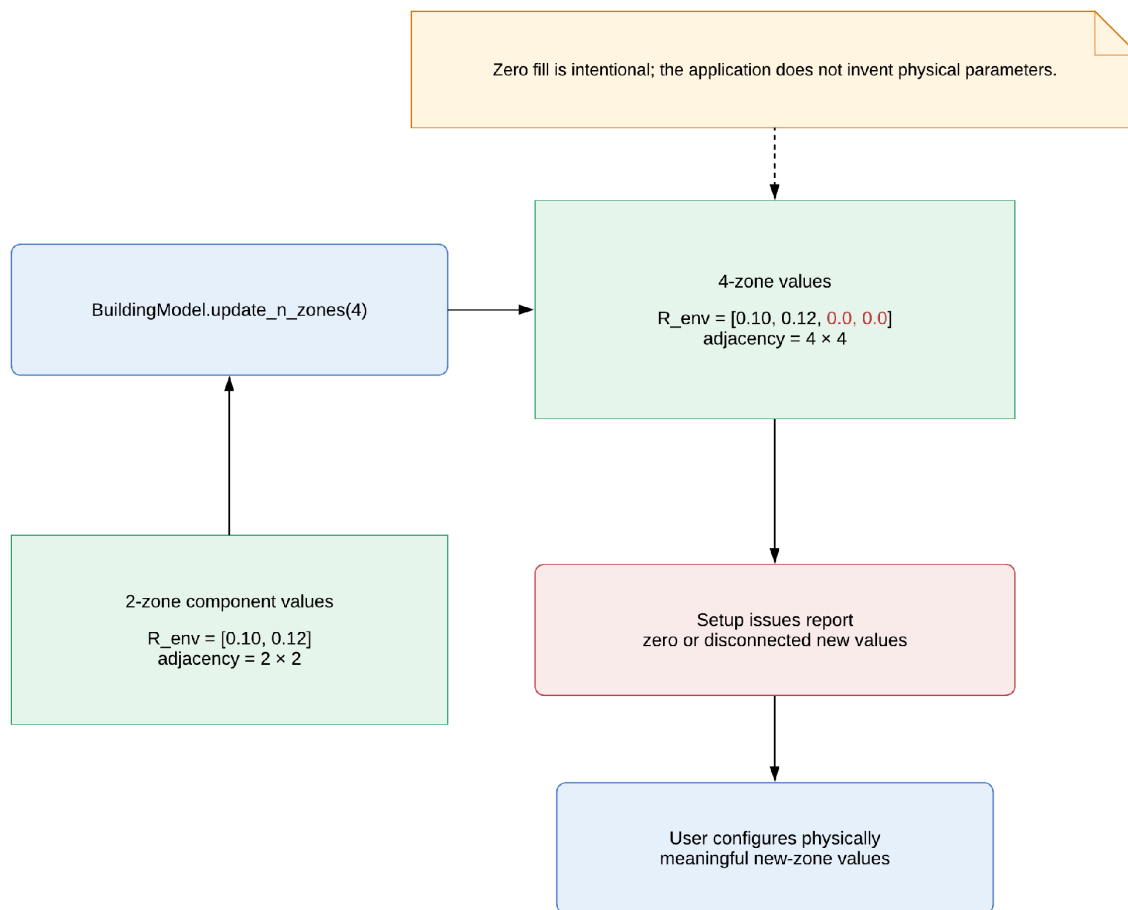


Figure: Zone Resize Example

Stable IDs Protect Saved Connections

Saved components receive IDs such as `component-0001`. Connections use these IDs because graphics-scene and list ordering are not durable identities. Legacy numeric indexes remain in the format as a compatibility fallback.

Simulation Data Flow

`SimulationRunner.run()` performs the following pipeline:

10. validate components and timing;
11. order nodes;
12. rebuild node input maps from connections;
13. build generated, file-based, and policy input tensors;
14. create a NeuroMANCER `BuildingSystem`;
15. run or resume the rollout; and
16. choose result variables for plotting.

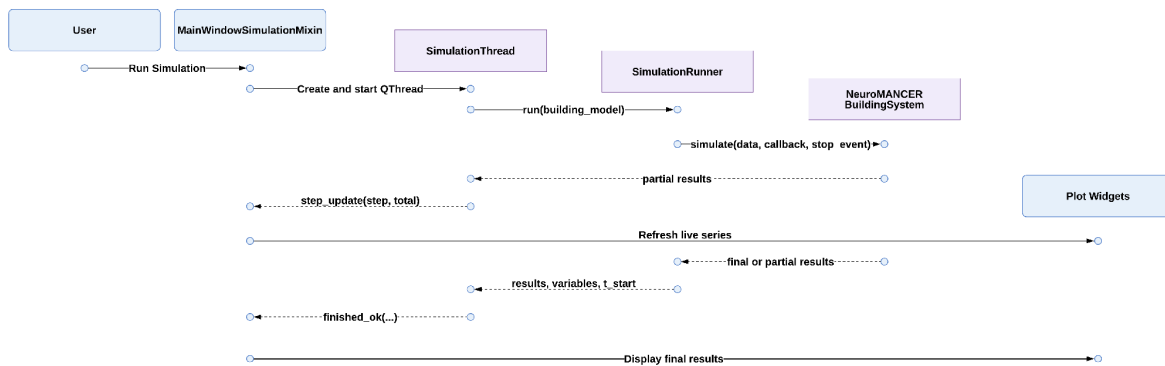


Figure: Simulation Sequence

Time and Tensor Contracts

Time settings use seconds. Requested times are:

```
range(t_start, t_start + t_duration, dt)
```

The end time is exclusive. Input and result series generally use:

```
[batch, time, features]
```

The GUI creates one batch. Scalar series use one feature; zone-dependent series use `n_zones` features.

Node Ordering

The runner first attempts a topological sort. HVAC systems may contain valid feedback cycles, so supported HVAC nodes use this deterministic fallback priority:

```
solar -> rtu -> vav -> envelope
```

Duplicate node names are rejected because backend result keys are namespaced by node name.

Input Precedence

Inputs are applied in this order:

17. component-generated schedules;

18. supported series loaded from CSV or SQLite; and
19. explicitly enabled project-level control-policy overrides.

Later layers replace earlier values. Default policy values do not override generated schedules until the override is enabled.

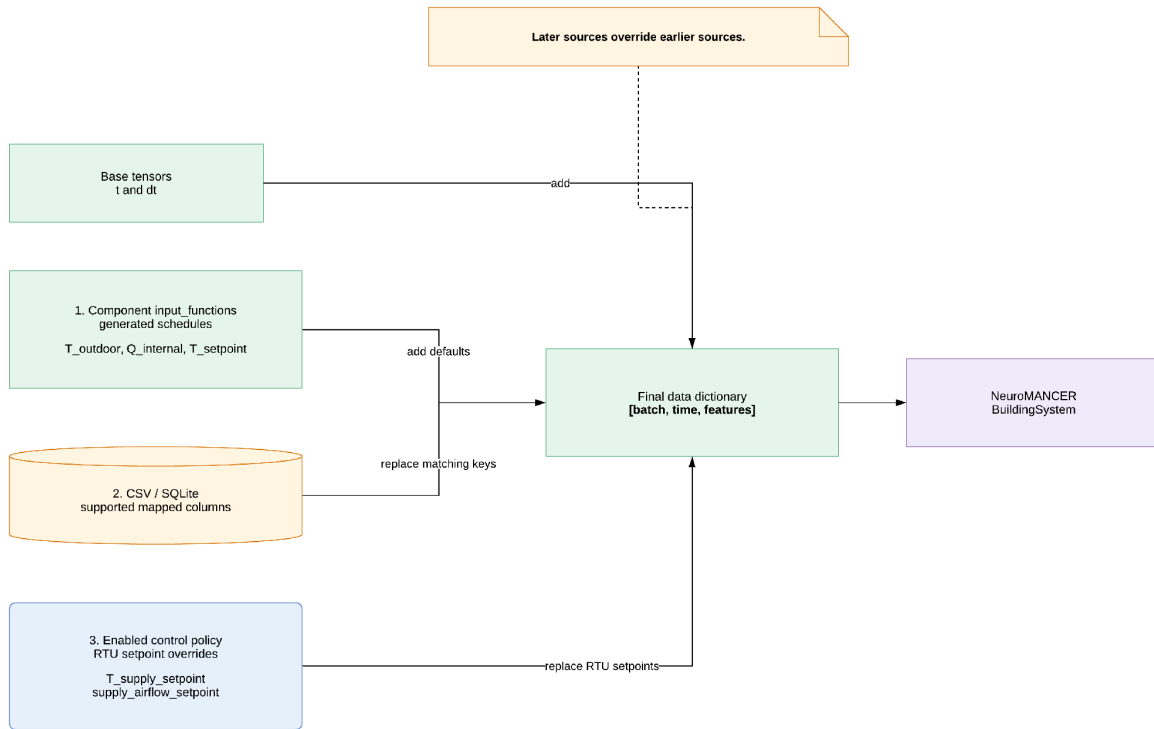
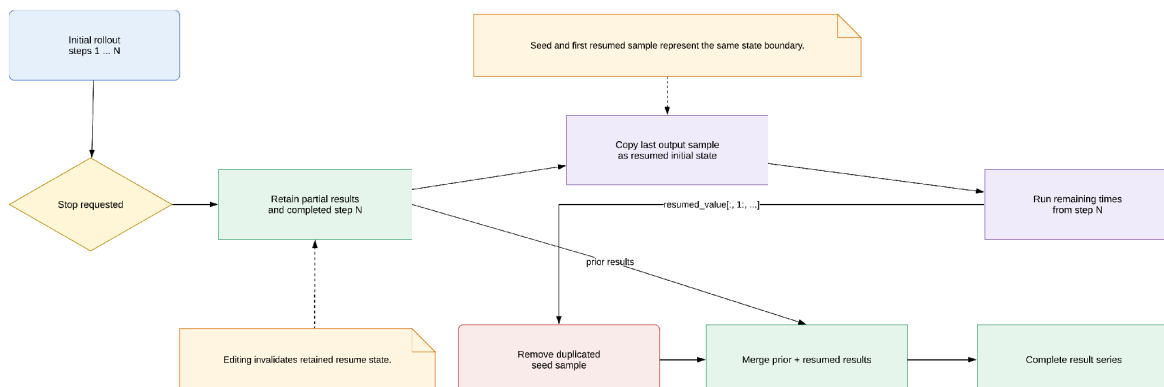


Figure: Input Precedence

Stop and Resume

Simulation runs in `SimulationThread` so Qt's event loop remains responsive. Stopping sets a cooperative `threading.Event`. Partial results are retained.

Resume seeds node outputs with their most recent samples, runs the remaining time range, and merges the new results with the prior results. The copied seed sample is excluded during merge to prevent a duplicate boundary timestep.



External Input Data

Supported input files are CSV and SQLite. Exact supported columns are:

File column	Resulting key	Conversion
Time [s]	Time coordinate	Used relative to project start
Outdoor Dry-Bulb Temperature [°C]	T_outdoor	Celsius to kelvin
Global Horizontal Irradiance [W/m²]	weather_factor	Clamped to 0..1 using 800 W/m²
AHU Supply Air Temperature [°C]	T_supply_setpoint	Celsius to kelvin
AHU Supply Fan Air Flow [m³/s]	supply_airflow_setpoint	Converted to kg/s using 1.2 kg/m³
Zone N Cooling Setpoint [°C]	T_setpoint	Celsius to kelvin

Values are linearly interpolated onto requested simulation times. Values outside the file range use the nearest endpoint. If fewer zone setpoint columns exist than project zones, the final available zone column is reused and a setup warning reports the mismatch.

Saved Project Format

Project JSON stores:

Field	Purpose
name	Project name
n_zones	Project-wide zone count
components	Ordered component records
component_sections	Component records keyed by stable ID
connections	Selected signal mappings and component references
time	Start, duration, and timestep
control_policy	Optional RTU override settings
input_data	External data path and descriptive summary

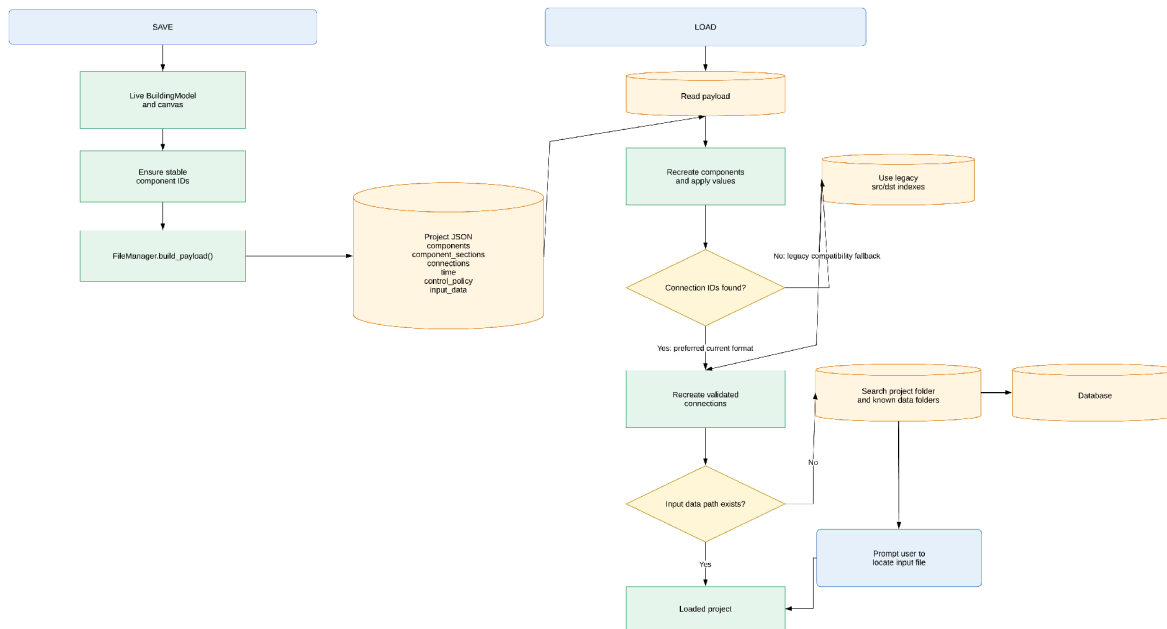


Figure: Project Save and Load Flow

Method Return Conventions

The following conventions apply throughout `src/`:

- Qt event handlers such as `mousePressEvent`, `resizeEvent`, and `paintEvent` perform side effects and return `None` unless Qt requires a base-class return.
- UI construction helpers return the widget or layout they create.
- UI synchronization helpers update existing widgets and return `None`.
- Dialog confirmation methods return `bool`.
- File prompt methods return a selected path or `None` when cancelled.
- Canvas validation methods commonly return `(success, message)` or `(success, message, connection_data)`.
- Simulation entry points return `(results, selected_variables, t_start)`.

Method Reference

`src/main.py`

Method	Arguments	Returns	Purpose
<code>main()</code>	None	Qt application exit code	Creates <code>QApplication</code> , displays <code>MainWindow</code> , and starts the event loop

The package initializer files `src/__init__.py`, `src/gui/__init__.py`, `src/models/__init__.py`, and `src/simulation/__init__.py` contain package descriptions and do not define callable APIs.

src/models/building_model.py

BuildingModel

Method	Arguments	Returns	Purpose
set_time_param_in_seconds	Optional t_start, t_duration, dt	None	Updates provided timing values as floats
set_control_policy	Optional supply temperature, airflow, and enable flag	None	Updates project-level RTU override settings
get_control_policy_data	None	dict	Returns serializable policy settings
_resize_zone_value	Value and target zone count	Resized value	Truncates or zero-pads vectors and matrices while preserving tensor dtype
update_n_zones	New zone count	None	Updates project and known zone-dependent component values
_infer_zone_count_from_value	Tensor or list	`int`	None` Infers zone count from vector or matrix dimensions
infer_n_zones_from_components	None	int	Returns the largest zone count found in known component properties
add_node, add_componentItem, add_connection	Object to add	None	Registers model graph objects
remove_node, remove_componentItem, remove_connection	Object to remove	None	Removes objects and dependent connections where applicable
run_simulation	None	(results, variables, t_start)	Runs the current project through SimulationRunner

src/simulation/input_data.py

Method	Arguments	Returns	Purpose
_to_float	Value and default	float	Converts input safely, returning the default on invalid data
_linear_interpolate	Times, values, target time	float	Interpolates within the series and clamps outside endpoints

Method	Arguments	Returns	Purpose
<code>_series_tensor</code>	Rows, column, target times, transform	<code>torch.Tensor</code>	Creates a scalar series shaped <code>[1, time, 1]</code>
<code>_zone_tensor</code>	Rows, zone columns, target times, zone count, transform	<code>torch.Tensor</code>	Creates a zone series shaped <code>[1, time, n_zones]</code>
<code>_read_csv_rows</code>	Path	<code>(rows, fieldnames)</code>	Reads CSV records and headers
<code>_read_database_rows</code>	Path	<code>(rows, columns)</code>	Reads the first supported SQLite table
<code>_read_input_rows</code>	Path	<code>(rows, columns)</code>	Selects CSV or database loading by suffix
<code>inspect_input_csv</code>	Path	<code>dict</code>	Reports path, row count, detected zones, and mapped keys
<code>load_input_csv</code>	Path, simulation times, zone count, start time	<code>(tensor_dict, summary_dict)</code>	Loads, converts, and interpolates supported external inputs

src/simulation/runner.py

SimulationRunner

Method	Arguments	Returns	Purpose
<code>run</code>	Model plus optional callback, stop, and resume state	<code>(results, variables, t_start)</code>	Validates, builds, executes, and optionally resumes a simulation
<code>_node_output_keys</code>	Nodes	<code>set[str]</code>	Collects all node output keys
<code>_seed_resume_data</code>	Input data, prior results, output keys	<code>set[str]</code>	Copies latest output states into resumed input data
<code>_merge_resume_results</code>	Prior results, resumed results, seeded keys	<code>dict</code>	Joins results without duplicating seed states
<code>_topological_sort</code>	Building model	<code>list</code>	Returns dependency order or deterministic HVAC priority
<code>_sort_by_hvac_priority</code>	Nodes	<code>list</code>	Sorts known HVAC nodes into backend execution order
<code>_validate_nodes</code>	Nodes	<code>None</code> ; raises <code>SimulationError</code>	Rejects duplicate backend node names
<code>_apply_connection_input_maps</code>	Building model	<code>None</code>	Rebuilds destination input maps from current connections
<code>_detect_control_policy</code>	Building model	Policy object or <code>None</code>	Finds connected or enabled project-level policy

Method	Arguments	Returns	Purpose
<code>_build_input_data</code>	Component items, times, optional model	<code>dict[str, Tensor]</code>	Builds generated, external, and override input tensors

SimulationThread

Method	Arguments	Returns	Purpose
<code>stop</code>	None	None	Requests cooperative simulation cancellation
<code>run</code>	None	None; emits Qt signals	Executes SimulationRunner and emits progress, result, or error signals

src/simulation/plotter.py

Method	Arguments	Returns	Purpose
<code>kelvin_to_celsius</code>	Numeric values	Converted values	Converts display temperatures
<code>auto_title</code>	Model name, timing, optional tab name	<code>str</code>	Builds a human-readable plot title
<code>select_variables</code>	Result dictionary	<code>list[str]</code>	Selects preferred plot variables or tensor-series fallback values

src/gui/canvas_items.py

ComponentItem

Method	Arguments	Returns	Purpose
<code>_non_overlapping_position</code>	Proposed scene position	<code>QPointF</code>	Finds a nearby position outside component spacing barriers
<code>_input_map_for</code>	Backend node name	<code>dict</code>	Returns the component's default input-key mapping
<code>pad_attribute</code>	Sequence and zone count	<code>list</code>	Truncates or repeats the final value to match zones
<code>pad_matrix</code>	Matrix and zone count	<code>list[list]</code>	Resizes a matrix and adds identity-style rows
<code>createComponent</code>	Display component name	<code>(component, BuildingNode)</code>	Creates the configured backend component and node
<code>edit_properties</code>	None	None	Opens a property dialog and applies accepted edits
<code>get_mutable_property_names</code>	None	<code>list[str]</code>	Returns editable properties for this component type

Method	Arguments	Returns	Purpose
<code>serialize_values</code>	None	<code>dict</code>	Converts editable component properties into JSON-safe values
<code>apply_serialized_values</code>	Value dictionary	None	Applies saved values using existing property types

`DragButton` and `ComponentItem` mouse, hover, paint, context-menu, and item-change handlers primarily update Qt state and return `None` or the required base-class value.

src/gui/interactive_canvas.py

Component operations

Method	Arguments	Returns	Purpose
<code>add_component</code>	Name, optional position, ID, values	<code>ComponentItem</code>	Adds an undoable component
<code>internal_add_component</code>	Same as above	<code>ComponentItem</code>	Performs component creation without pushing undo
<code>create_drag_preview_item</code>	Name and position	Preview <code>ComponentItem</code>	Creates a non-model drag preview
<code>drag_preview_position</code>	View position	<code>QPointF</code>	Converts cursor position to centered scene position
<code>delete_component_items</code>	Component items	<code>int</code>	Deletes valid unique items and returns count
<code>set_area_delete_preview</code>	Scene rectangle	<code>list[ComponentItem]</code>	Highlights and returns components intersecting an area

Connection operations

Method	Arguments	Returns	Purpose
<code>available_connection_mappings</code>	Source and destination items	Mapping list or <code>None</code>	Returns supported signal mappings for the pair
<code>_resolve_connection_mappings</code>	Available, selected, and legacy mapping fields	Mapping list	Resolves the mappings to apply
<code>add_connection_between_items</code>	Source, destination, optional mappings	Normally <code>(bool, str)</code> ; direct mode returns <code>(bool, str, connection_data)</code>	Adds an undoable validated connection, or delegates directly when no undo stack exists
<code>internal_add_connection_between_items</code>	Same as above	<code>(bool, str, connection_data)</code>	Performs validated connection creation

Method	Arguments	Returns	Purpose
<code>add_connection_between_selected</code>	None	(bool, str)	Connects exactly two selected components
<code>remove_connection_data</code>	Connection data and notify flag	bool	Removes through undo when available
<code>internal_remove_connection_data</code>	Connection data and notify flag	bool	Removes visual and logical connection data directly
<code>delete_connection_data</code>	Connection data and notify flag	bool	Deletes a connection through the configured workflow
<code>_connection_at_view_pos</code>	View position	Connection data or None	Finds the nearest clickable connection

Canvas zoom, drawing, highlighting, geometry, drag/drop, and Qt event methods update view state and return `None` unless they are hit-testing helpers.

[src/gui/canvas_tool_manager.py](#)

Method	Arguments	Returns	Purpose
<code>get_zoom_percent</code>	None	int	Returns current canvas zoom percentage
<code>nearest_button_zoom_percent</code>	None	int	Returns nearest predefined button zoom step
<code>minimum_zoom</code>	None	float	Calculates minimum zoom allowed by viewport and scene
<code>viewport_center</code>	None	QPointF	Returns current scene-space viewport center
<code>set_zoom_factor</code>	Requested zoom	Applied float	Clamps and applies an absolute zoom factor
<code>_next_button_zoom_step</code>	Direction	float	Calculates accelerated button zoom increment
<code>handle_mouse_press_event</code>	Qt event	bool	Handles pan/area modes and reports whether consumed
<code>handle_mouse_move_event</code>	Qt event	bool	Handles active pan/area movement and

Method	Arguments	Returns	Purpose
			reports whether consumed
handle_mouse_release_event	Qt event	`None`	int "handled"
			Reports no action, deleted count, or handled pan release

Other methods mutate canvas tool state and return `None`.

src/gui/file_manager.py

Method	Arguments	Returns	Purpose
<code>_should_inline_list</code>	Value	<code>bool</code>	Decides whether a JSON list is readable on one line
<code>_format_json_with_inline_lists</code>	Value and indent level	<code>str</code>	Produces valid readable JSON
<code>get_saved_dir</code>	Last directory	<code>Path</code>	Returns preferred layout-save directory
<code>get_input_data_dir</code>	Optional last directory	<code>Path</code>	Returns preferred external-data directory
<code>get_input_data_search_dirs</code>	Optional layout path	<code>list[Path]</code>	Returns deduplicated search locations
<code>resolve_input_data_path</code>	Saved path and optional layout path	<code>`Path`</code>	<code>None`</code> Resolves a portable external-data reference
<code>build_payload</code>	Project state values	<code>dict</code>	Builds the complete serializable project payload
<code>save_layout</code>	Project state and destination	Destination path	Writes project JSON
<code>load_payload_from_path</code>	Path	<code>dict</code>	Reads project JSON
<code>get_model_name,</code> <code>get_n_zones,</code> <code>get_components,</code> <code>get_component_sections,</code> <code>get_connections,</code> <code>get_time_data,</code> <code>get_control_policy_data,</code> <code>get_input_data</code>	Payload and occasional default	Field value	Safely reads a project-format section

src/gui/dialogue_manager.py

Method	Arguments	Returns	Purpose
confirm_load_project	None	bool	Confirms replacing a project with unsaved changes
confirm_exit	None	bool	Confirms closing with unsaved changes
confirm_new_project	Unsaved-change flag	bool	Confirms project reset when needed
confirm_delete_items	Labels	bool	Confirms area deletion
prompt_connection_mappings	Available mappings	Selected mapping list or None	Lets the user choose signals
prompt_load_layout_path	Start directory	Path or None	Prompts for project JSON
prompt_input_data_path	Start directory	Path or None	Prompts for CSV or SQLite data
prompt_save_layout_path	Start directory	Path or None	Prompts for project destination

Dialog constructors build UI state. Their `accept()` methods validate or apply values, call the Qt base implementation, and return `None`.

src/gui/main_window_project.py

Method	Arguments	Returns	Purpose
_generate_component_id	None	str	Creates the next stable component ID
_extract_component_id_number	ID	int	None` Parses generated numeric suffix
_area_delete_item_labels	Items	list[str]	Builds confirmation labels
confirm_area_delete_items	Items	bool	Requests deletion confirmation
_is_pristine_empty_new_project	None	bool	Detects unchanged default project
_has_unsaved_changes	None	bool	Reports meaningful dirty state
_confirm_new_project	None	bool	Confirms project reset
save_as_layout	None	Saved path string or False	None` Prompts and saves
save_layout	None	Saved path string or delegated result	Saves to current path or invokes Save As

Method	Arguments	Returns	Purpose
load_layout	None	`True` False None`	Loads a selected project or reports cancellation

Tool arming, action-mode handlers, refresh callbacks, dirty-state updates, reset, and component/connection callbacks perform UI and project side effects and return `None`.

src/gui/main_window_simulation.py

Method	Arguments	Returns	Purpose
_completed_simulation_steps	Result dictionary	int	Infers completed backend steps while ignoring full-length external series
run_simulation	None	None	Validates UI state and starts SimulationThread
stop_simulation	None	None	Requests cooperative stop
resume_simulation	None	None	Starts a thread from retained partial results
_on_sim_step_update	Step and total	None	Refreshes progress and live plots
_on_sim_finished_ok	Results, variables, start	None	Stores final or partial results and updates simulation UI
_on_sim_error	Message	None	Resets simulation UI and displays failure

Cache, view, progress, and button-state helpers update UI state and return `None`.

src/gui/main_window_ui.py

Method	Arguments	Returns	Purpose
_window_ui_scale	None	float	Calculates responsive scale
_canvas_component_count	None	int	Counts real canvas components
_button_for_component_action_mode	Mode	Button or <code>None</code>	Finds the active tool button
create_left_panel, create_right_panel, create_project_tab, _create_simulation_tab	None	QWidget	Constructs major UI regions
_create_info_box, _create_zone_controls_box, _create_zoom_controls_box, _create_no_sim_placeholder	Relevant label or none	Widget	Constructs reusable UI sections

Method	Arguments	Returns	Purpose
<code>_zone_value_shape</code>	Value	Shape tuple	Normalizes list/tensor shape inspection
<code>_numeric_values</code>	Value	Numeric list	Flattens supported values for validation
<code>_matrix_rows</code>	Value	Matrix rows	Normalizes a matrix for validation
<code>collect_setup_issues</code>	None	<code>list[str]</code>	Returns current configuration warnings

Other UI methods synchronize controls, change project settings, refresh labels, or load/clear external data and return `None`.

src/gui/state_manager.py

Method	Arguments	Returns	Purpose
<code>_format_property_value</code>	Value	<code>str</code>	Formats nested values for display
<code>_connection_item_name</code>	Item and fallback	<code>str</code>	Returns a readable component name
<code>refresh_component_list</code> , <code>refresh_connection_list</code> , <code>update_zone_display</code>	None	<code>None</code>	Rebuilds project-state displays

`Connection` is a simple logical record containing source node/output and destination node/input.

src/gui/undo_commands.py

All undo command constructors capture enough state to reverse their operation. `redo()` performs or restores the mutation; `undo()` reverses it. These methods return `None`.

Command	Captured operation
<code>AddComponentCommand</code>	Add and restore one component
<code>DeleteComponentCommand</code>	Remove one component and its connections
<code>AddConnectionCommand</code>	Add and restore one connection
<code>DeleteConnectionCommand</code>	Remove and restore one connection
<code>DeleteComponentsCommand</code>	Atomically remove and restore a component subgraph
<code>MoveComponentCommand</code>	Move between recorded start and end positions

src/gui/plot_widget.py

Method	Arguments	Returns	Purpose
<code>_padded_range</code>	Values, padding fraction, minimum padding	<code>(low, high)</code>	Creates a readable numeric axis range
<code>_time_domain_with_margin</code>	Time vector	<code>(low, high)</code>	Creates a padded time range
<code>LineSeries.visible</code>	None	<code>bool</code>	Reports series visibility
<code>ChartWidget.set_y_axis_range</code>	Minimum and maximum	<code>bool</code>	Applies a valid manual range and reports success
<code>ChartWidget._nearest_sample</code>	Cursor and plot rectangle	Sample information or <code>None</code>	Finds nearest visible plotted sample
<code>ChartWidget._nice_ticks</code>	Range and desired count	<code>list[float]</code>	Produces readable axis ticks
<code>ChartWidget.render_to_pixmap</code>	Width, height, scale	<code>QPixmap</code>	Renders a chart for export

Chart configuration, drawing, zooming, hover, and mouse handlers update visual state and return `None` unless listed above.

src/gui/plot_containers.py

Method	Arguments	Returns	Purpose
<code>MultiChartWidget._elapsed_time_vector</code>	Results	<code>numpy.ndarray</code>	Returns elapsed plot time
<code>MultiChartWidget._series_color</code>	Metadata and feature information	<code>QColor</code>	Chooses series color
<code>MultiChartWidget.load_results</code>	Results, variables, timing, metadata, colors	<code>list[ChartWidget]</code>	Rebuilds and returns charts
<code>MultiChartWidget._chart_insert_idx</code>	Vertical coordinate	<code>int</code>	Finds drag-reorder insertion index
<code>MultiChartWidget.save_to_file</code>	Path and render sizing	<code>None</code>	Exports all charts
<code>VarListWidget.populate</code>	Variable keys and metadata	<code>None</code>	Rebuilds available-variable list

Remaining methods update chart appearance, live series, drag/drop order, or widget dimensions and return `None`.

Simple GUI Modules

Module	Important contracts
<code>main_window.py</code>	Constructs and wires the complete application window; methods are side-effect-only
<code>main_window_plots.py</code>	Plot-tab creation helpers return widgets/icons; plot actions update or export charts

Module	Important contracts
<code>main_window_helpers.py</code>	Style helpers return stylesheet strings; <code>_plot_settings_title</code> returns a shortened label
<code>header_bar.py</code>	Button factory methods return configured buttons; other methods build or restyle the toolbar
<code>icons.py</code>	<code>IconProvider.icon()</code> returns cached, located, or generated <code>QIcon</code> ; search helpers return paths or <code>None</code>
<code>results_window.py</code>	<code>ResultsWindow</code> constructs a dialog for an existing figure

Current Constraints

- Runnable simulations require unique component node names, effectively one component of each supported type.
- Graph edges are intentionally limited by `HVAC_CONNECTIONS`.
- New zone-dependent values may require user configuration after resizing.
- External input columns are matched by exact name.
- Simulation stopping is cooperative and occurs between backend steps.

Appendix: Simulation Value Reference

Inputs, Setpoints, and Temperatures

Value	Reasonable range	Meaning / why it can be negative
t	simulation start to end, seconds	Time vector. Should not be negative unless the user starts the simulation at a negative time.
T_outdoor	about -20 to 45 C	Outdoor dry-bulb temperature. Displayed in Celsius; negative Celsius is normal winter weather. Internally it is Kelvin and should never be negative.
weather_factor	0 to 1	Cloud/irradiance factor. 0 means no useful sun, 1 means full design irradiance. Should not be negative.
Q_internal	0 to 2000 W per zone	Heat from people, lights, equipment. Should not be negative because it represents heat added to a zone.
solar.Q_solar	0 to 10000 W per zone	Solar heat through windows. Should not be negative; night/clouds reduce it toward zero.
T_setpoint	about 15 to 30 C	Zone temperature target. Can be below 0 C only with bad/custom input; normal HVAC setpoints are positive Celsius.
T_supply_setpoint	about 10 to 25 C	RTU supply-air temperature target. Should normally be positive Celsius.
supply_airflow_setpoint	0 to 5 kg/s	Desired RTU airflow. Should not be negative; negative airflow would be physically invalid here.
envelope.T_zones	about 10 to 50 C	Simulated zone temperatures. Negative Celsius is possible mathematically if the model/input is extreme, but it usually means unrealistic weather, loads, HVAC, timestep, or envelope parameters.
rtu.T_supply	about 5 to 50 C	RTU discharge/supply-air temperature. Should not be negative Celsius in normal operation, though the internal clamp allows a wide Kelvin range.
vav.T_supply	about 10 to 50 C	Final VAV supply temperature after possible reheat. Should normally stay between cold supply air and warmed/reheated air.

Airflow, Pressure, and Controller State

Value	Reasonable range	Meaning / why it can be negative
rtu.supply_airflow	0 to 5 kg/s	Actual RTU supply airflow. Should not be negative because damper position is clamped 0..1.
vav.supply_airflow	0 to 2 kg/s per zone	Actual VAV airflow. Should not be negative.
rtu.P_supply	about 100000 to 105000 Pa	RTU supply pressure. Should not be negative.
vav.P_supply	about 200 to 2000 Pa	VAV downstream pressure. Should not be negative in a valid setup; it is upstream duct pressure minus pressure drop.
rtu.damper_position	0 to 1	RTU airflow damper fraction. Cannot be negative in normal operation.
rtu.valve_position	0 to 1	RTU coil valve fraction. Cannot be negative in normal operation.
vav.damper_position	0 to 1	VAV damper fraction. Cannot be negative in normal operation.
vav.reheat_position	0 to 1	Electric reheat actuator fraction. Cannot be negative in normal operation.
rtu.integral_accumulator	-1 to 1	PI controller integral state. This can be negative because it stores signed accumulated temperature error: negative usually means cooling demand, positive usually means heating demand.

Power and Heat Flow

Value	Reasonable range	Meaning / why it can be negative
rtu.fan_power	0 to 10000 W	Electrical fan power. Should not be negative.
rtu.cooling_power	0 to 10000 W	Compressor electrical power used for cooling. The cooling heat transfer is signed internally, but the reported power is positive consumption.
rtu.heating_power	0 to 30000 W	Heating electrical/fuel power. Should not be negative.
rtu.total_power	0 to 50000 W	Sum of RTU fan, cooling, and heating power. Should not be negative.
rtu.supply_heat_flow	roughly -50000 to 100000 W, but code currently computes mostly positive enthalpy-like flow	Listed as a thermal supply stream value. Signed heat-flow convention would allow negative for cooling, but the current RTU formula uses $\text{airflow} * \text{cp} * T_{\text{supply_final}}$, so it should normally be positive.
vav.Q_supply_flow	-10000 to 50000 W per zone	Heat delivered from VAV air to the zone, calculated from $\text{airflow} * \text{cp} * (T_{\text{supply}} - T_{\text{zone}})$. Negative is expected when supply air is colder than the zone, meaning cooling/removing heat. Positive means heating/adding heat.
vav.total_power	0 to 5500 W	Electric reheat power. Should not be negative.